

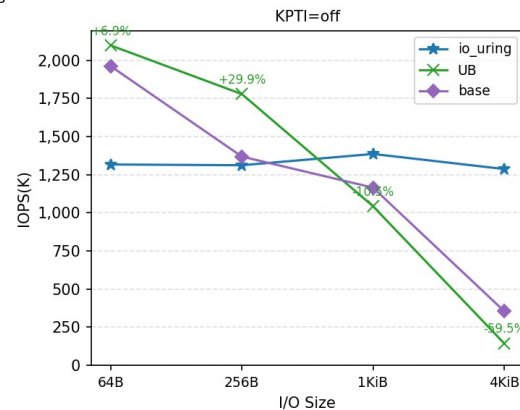
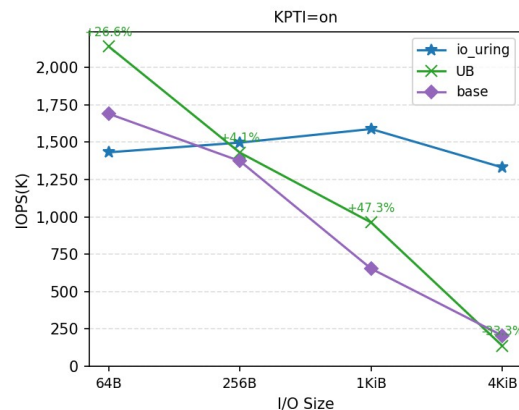
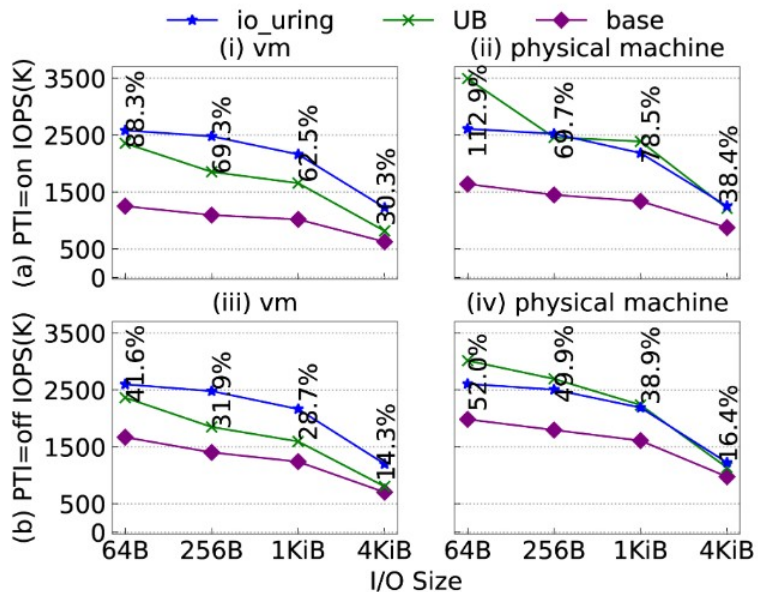
- Ubuntu 20.04.2 auf Oracle VirtualBox eingerichtet
- VirtualBox Eigenschaften:
 - Hauptspeicher: 4096 MB
 - CPUs: 2
- Für Experiment nötige Software eingerichtet (Qemu 4.2.1, Redis 6.2.6, Nginx 1.20.0)
- Kernel 5.4.44 gedownloadet und mithilfe der Patch File aus dem Repo Kernel gepatched
- Kernel kompiliert und als Boot Kernel eingerichtet

- Experimente im Paper wurden jeweils mit und ohne KPTI ausgeführt
- KPTI deaktivierung:
 - nano /etc/default/grub
 - nopti zu GRUB_CMDLINE_LINUX_DEFAULT="" hinzufügen
 - Grub updaten, Ubuntu neu starten

- Address Randomization ausschalten: `sudo su → echo 0 > /proc/sys/kernel/randomize_va_space`
- Kompilieren des daemon Programms mit `make`
- Sudo `./start.sh` zum starten von UB
- Programm laufen lassen
- Deinstallierung mit `sudo rmmod zz_lkm`

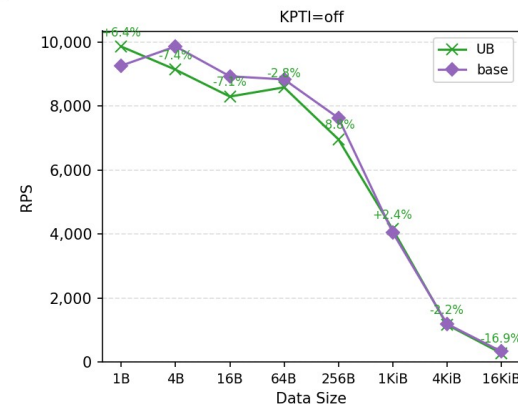
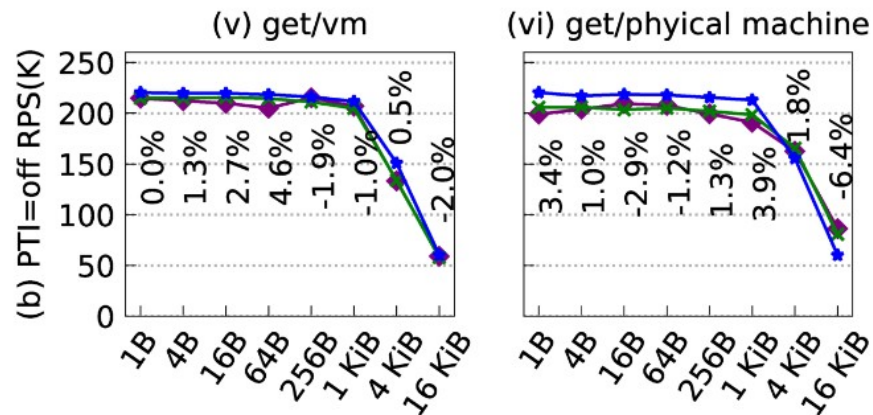
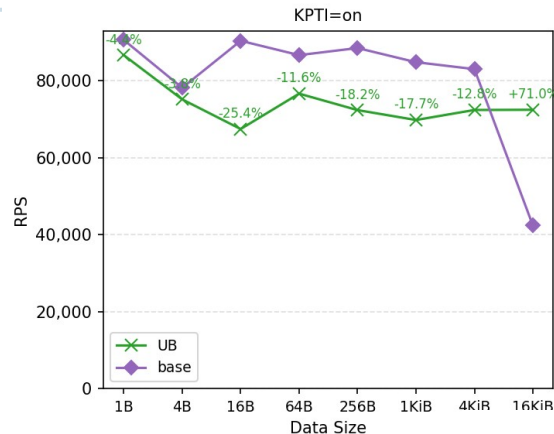
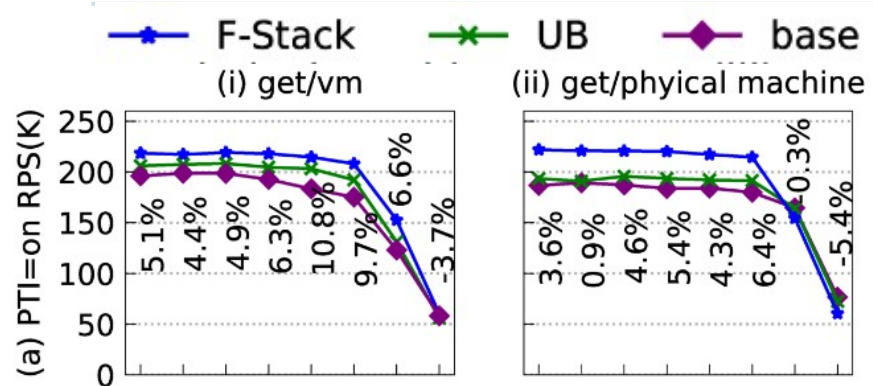
- Syscall read:
 - Erstellung test.file: `dd if=/dev/zero of=test.file bs=1M count=2048`
 - `Make & sudo ./syscall_read <IO_SIZE>` mit jeweiliger Angabe der I/O Size
 - Probleme: Pfad von Test_File in `syscall_read` wurde nicht gefunden, musste angepasst werden
- IO_Uring:
 - Einrichtung von fio 3.16
 - Durchführung mit fio, I/O Size und korrespondierende File Size mussten pro Benchmark angepasst werden

I/O Benchmark - Vergleich

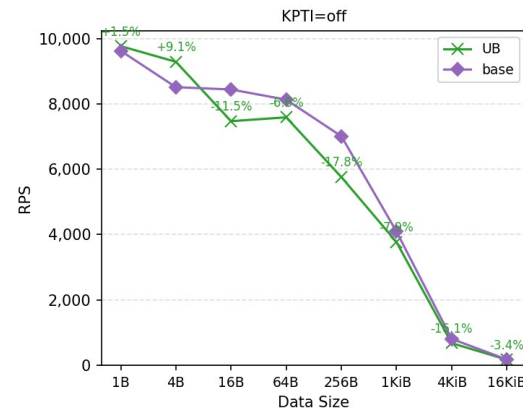
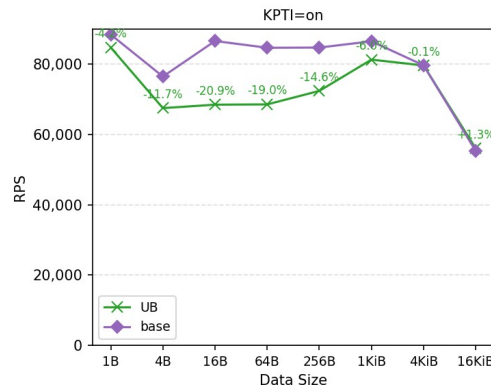
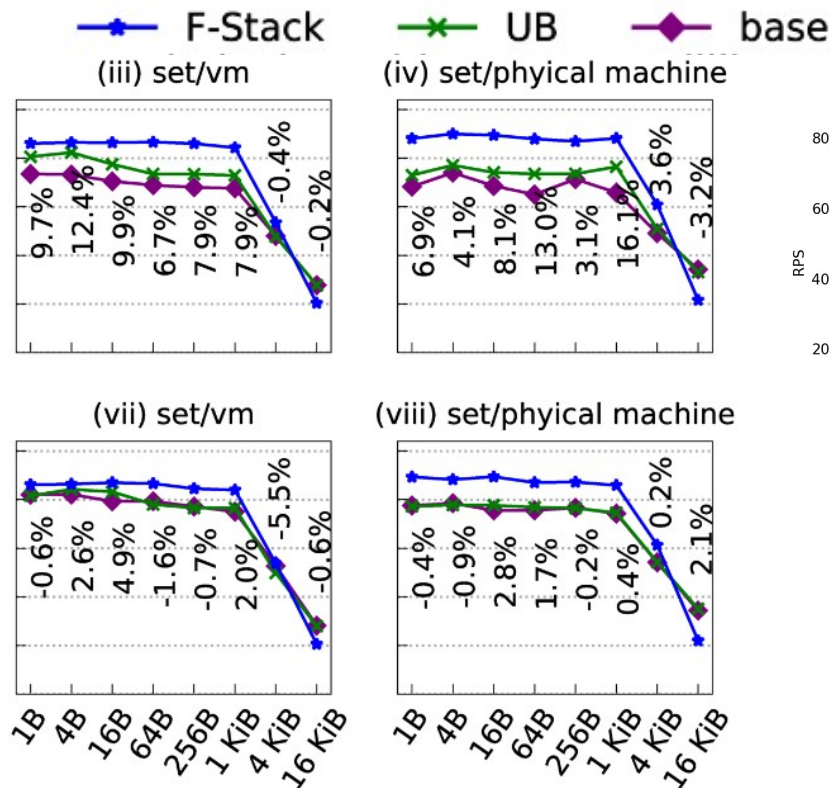


- Redis Server mit NIC verknüpfen
- Redis-Server auf Host starten: `./redis-server ../redis-conf`
- Auf Client Benchmark laufen lassen: `./redis-benchmark -h <IP_ADDRESS_OF_REDIS_SERVER> -p <PORT_OF_REDIS_SERVER> -t get -n 1000000 -d 3 --threads 2 (get oder set)`
- Problem: Host und Client könnten auf selber Maschine laufen, aufgrund schlechten Laptops wurde seperater Client genutzt. Beide VB hatten selbe IP Adresse. Durch Einstellungen in VB gefixt.
- F-Stack:
 - Seperater Benchmark
 - Problem: Lies sich nicht ausführen

Redis Benchmark – Vergleich: Get

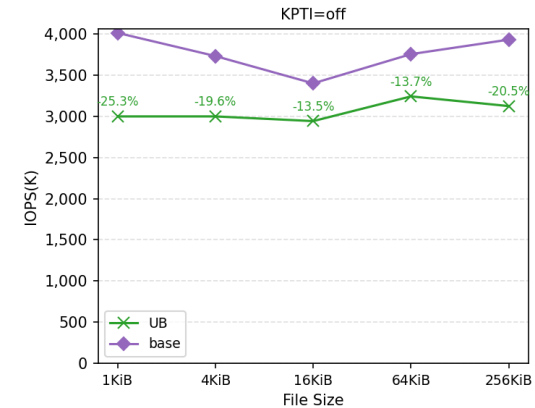
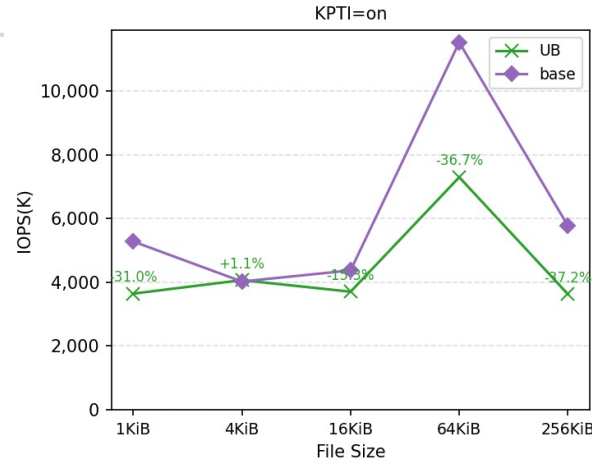
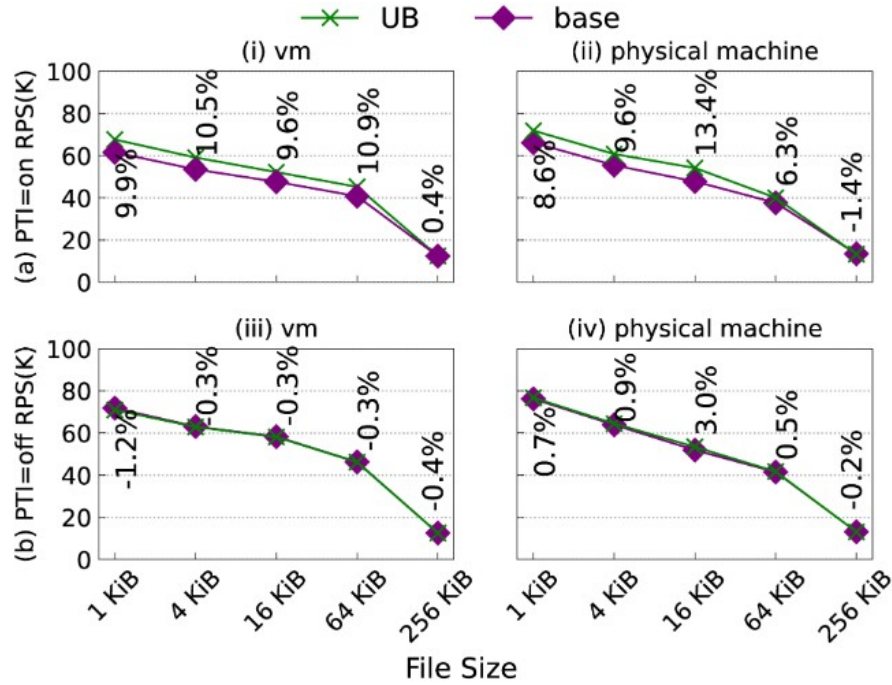


Redis Benchmark – Vergleich: Set

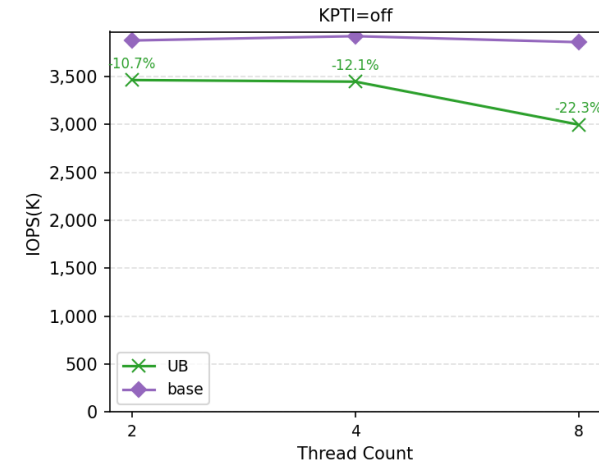
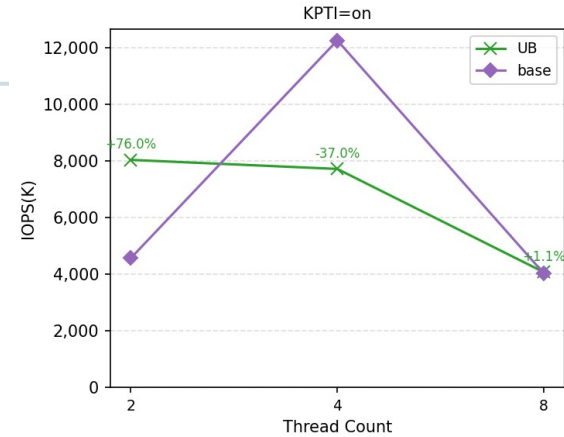
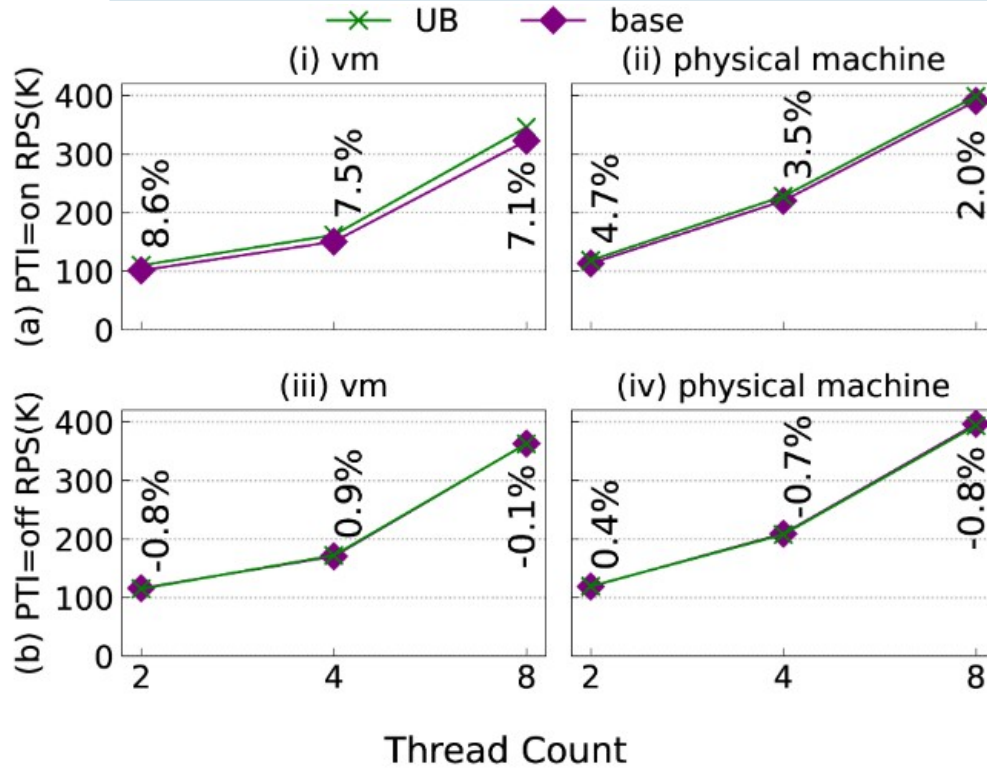


- Benchmark durch wrk auf seperater VB
- Generierung Datei für Benchmark: `sudo dd if=/dev/zero of=<FileSize>.html bs=<FileSize> count=1` für jede Dateigröße im Benchmark
- Start Nginx Server: `sudo nignx`
- Client: `./wrk -t8 -c1024 -d12 <URL>`

Nginx Benchmark – Vergleich verschiedene Dateigrößen

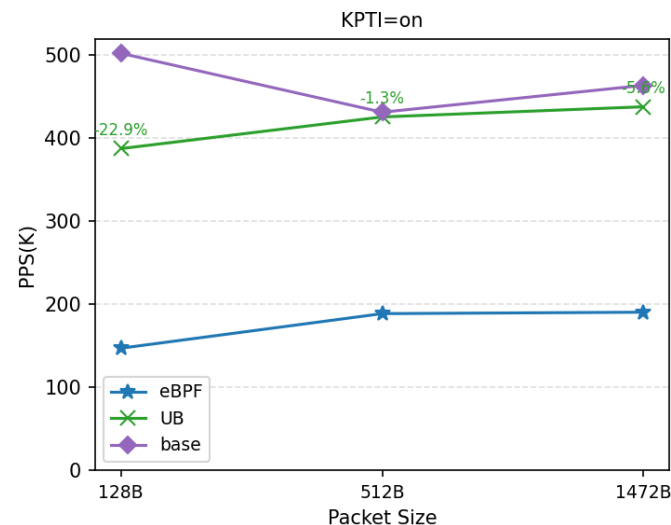
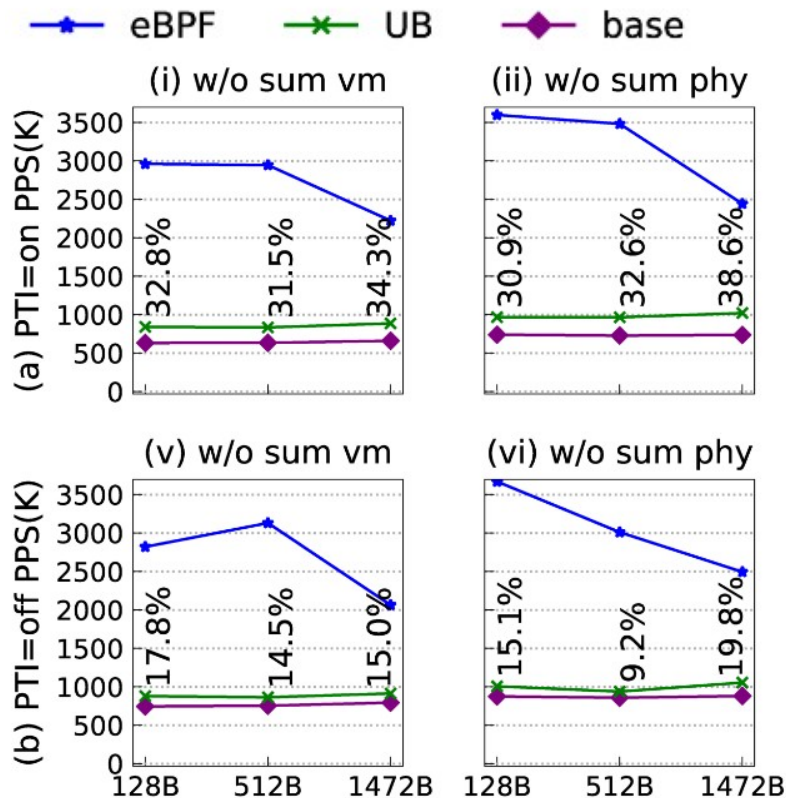


Nginx Benchmark – Vergleich verschiedene Threads



- Raw Socket:
 - Server: In `raw_socket_udp.c` Name der gewählten NIC angeben, kompilieren und Server mit `sudo ./sniff` starten
 - Client: In `send_udp.c` NIC Adresse des Hosts angeben, kompilieren und mit `./send_udp` verschicken
- EBPF:
 - Installation `bpfcc` und `bcc`
 - Server: In `source_codes/apps/socket/bpf/main.py` Namen des gewählten NIC angeben und mit `sudo python3 wrapper.py` starten
 - Client: Identisch zu Raw Socket

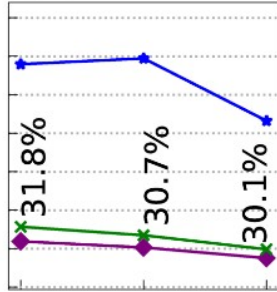
Raw Socket Benchmark – Vergleich without sum



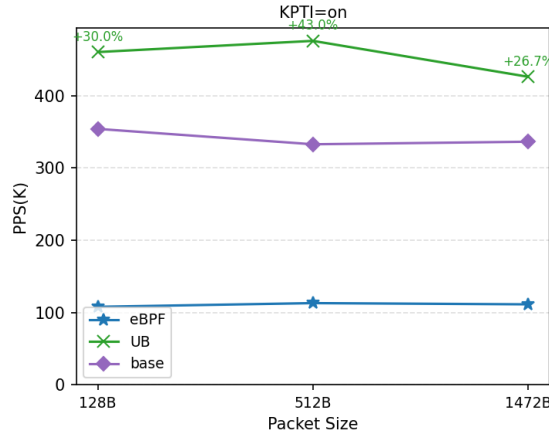
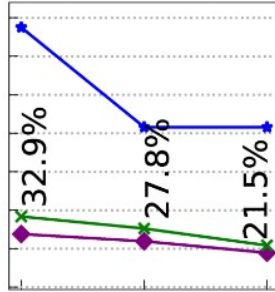
Raw Socket Benchmark – Vergleich with sum

—◆— eBPF —×— UB —◆— base

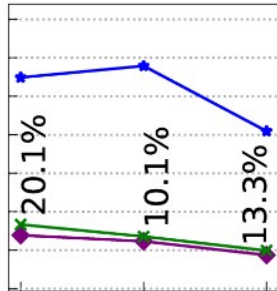
(iii) w/ sum vm



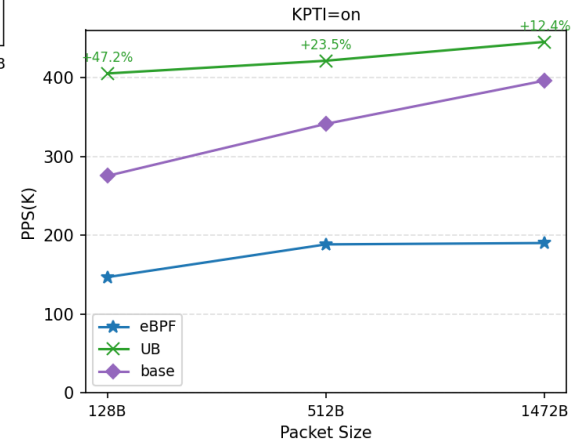
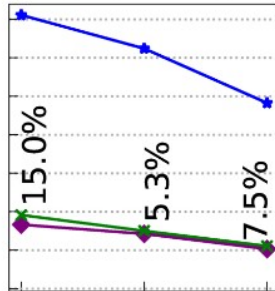
(iv) w/ sum phy



(vii) w/ sum vm



(viii) w/ sum phy



- Lange Dauer für Kernel-Kompilierung und Benchmarks (zT 3 Stunden)
- Probleme bei Installation benötigter Programme (lange Downloadzeiten, zufällige Download-abbrüche)
- ZT schlechte Dokumentation (Befehle ohne angegeben Datei Pfad, Unschlüssige Reihenfolgen)

- Im Paper: deutliche Verbesserung durch UB beim I/O Benchmark und Raw Socket, bei Redis und Nginx leichte Verbesserung
- Unsere Experimente: schwankende Verbesserungen bei I/O Benchmark und deutliche Verbesserungen bei Raw Socket (with sum)
- UB bei Redis mit KPTI schlechter, ohne KPTI ähnlich
- UB bei Nginx schlechter